



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №6
«Шаблони «Abstract Factory », «Factory Method », «Momento»,
«Observer», «Decorator ».»
Варіант - 4

Виконала:
Студентка групи ІА-22
Третякова Ельвіра

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми.....	8
Реалізація шаблону за обраною темою.....	10
<i>Діаграма класів для паттерну Decorator.....</i>	<i>14</i>
<i>Проблема, яку допоміг вирішити шаблон Decorator.....</i>	<i>15</i>
<i>Переваги застосування паттерну Decorator</i>	<i>16</i>
<i>Посилання на GitHub</i>	<i>17</i>
Висновок	17

Тема: шаблони «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator».

Мета: ознайомитися з шаблонами проектування «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

Варіант – 4: Графічний редактор (proxy, prototype, decorator, bridge, flyweight, SOA)
Графічний редактор повинен вміти створювати / редагувати растрові (або векторні на розсуд студента) зображення в 2-3 основних популярних форматах (bmp, png, jpg), мати панель інструментів для створення графічних примітивів, вибору кольорів, нанесення тексту, додавання найпростіших візуальних ефектів (ч/б растр, інфрачервоний растр, 2-3 на вибір учня), роботи з шарами.

Короткі теоретичні відомості

Принципи проектування SOLID

Принципи проектування SOLID були введені Робертом Мартіном в його статті "Об'єктно-орієнтоване проектування" і відносяться до розробки класів систем.

Принцип єдиної відповідальності (single responsibility principle) свідчить про необхідність створення класів, що відповідають не більше ніж за одну річ. Проблема, яка виникає коли один клас містить безліч обов'язків, - раніше всього, погана читаність коду (як правило, такі класи розростаються до неймовірних розмірів і стає практично неможливою їх подальша підтримка), і складність зміни. Наприклад, якщо служба звітів тісно інтегрована із службою друку (у одному класі), то зміни в одній із служб напевно приведуть до поломки інших. Цей принцип також веде до зменшення зв'язаності компоненту (low coupling) і підвищення цілісності елементів системи (high cohesion).

Принцип відкритості/закритості проголошує необхідність додавання можливості розширення без зміни початкових кодів самого компонента. Таким чином існує можливість зміни поведінки компонента без зачіпання початкових кодів, що може знадобитися у разі тестування (заміна окремих компонент заглушками) або додавання функціональності. Однією з реалізацій принципів відкритості/закритості є реалізація інтерфейсів або абстрактних класів для специфікації семантики необхідних дій (контракт послуг, що надаються), і надання можливостей до власної реалізації цих інтерфейсів (конкретна реалізація).

Принцип підстановки Барбари Лисков стверджує, що підкласи в ієрархії класів повинні зберігати функціональність, аналогічну батьківським класам. Підстановка підкласів повинна залишити програму функціонувати тим же чином (коректно). Проблема походить з наступних міркувань: припустимо, реалізується клас калькулятор; створюється дочірній клас "цифровий калькулятор", але в перевантаженні методу "сума" використовується множення. Синтаксично, таке використання дочірніх класів не несе помилок (компілюється і запускається). Семантично, результат роботи програми абсолютно поміняється при заміні батьківського класу підкласом (кожного разу при підсумовуванні ми будемо насправді множити). Принцип свідчить про необхідність зберігати семантику базового класу в уникнення безглузвих помилок в програмному забезпеченні.

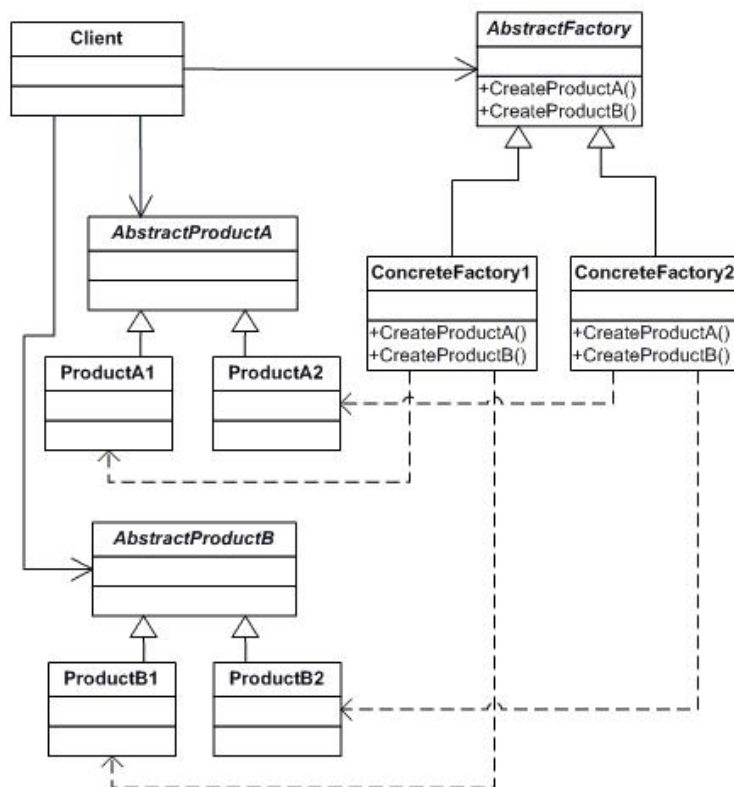
Основною ідеєю *принципу розділення інтерфейсів* є розбиття загального "контракту" (чи

набору послуг, що надаються) програми на безліч дрібних інтерфейсних модулів, які відповідають за одну конкретну роботу. Такий підхід нагадує принцип одного обов'язку, але відноситься до усієї системи в цілому, і спрощує читання, розуміння і модифікацію системи, оскільки користувачі інтерфейсу бачать лише той "розріз" системи, який їм в даний момент потрібний.

Принцип інверсії залежностей стверджує про необхідність усунення залежностей модулів верхнього рівня від модулів нижнього рівня, оскільки і ті і інші повинні посилатися на абстракції (інтерфейси). Абстракції, у свою чергу, не повинні залежати від деталей, а навпаки. Найпростіше зрозуміти цей принцип на конкретному прикладі: нехай існує деяка служба, що виробляє конкретні розрахунки (назвемо її Служба 1); існує служба, яка має посилання на неї, і використовує для реалізації власної внутрішньої механіки (назвемо її служба 2). Реалізація цього принципу виглядатиме таким чином: у Служби 1 буде винесений загальний інтерфейс, і цей інтерфейс використовуватиметься в Службі 2 (підставлятися в конструктор, наприклад). Таким чином, служба 1 залежить від абстракції (загальний інтерфейс), і не впливає на роботу Служби 2 (модулі верхнього рівня не залежать від модулів нижнього рівня). Абстракція у свою чергу не залежить від деталей, оскільки є простою специфікацією можливих дій. Слід зазначити, що цей принцип добре поєднується з попередніми, і дозволяє складати чистіші застосування.

Шаблон «Abstract Factory»

Структура:



Призначення патерну:

Шаблон "абстрактна фабрика" використовується для створення сімейств об'єктів без вказівки їх конкретних класів. Для цього виноситься загальний інтерфейс фабрики (**AbstractFactory**) і створюються його реалізації для різних сімейств продуктів. Хорошим

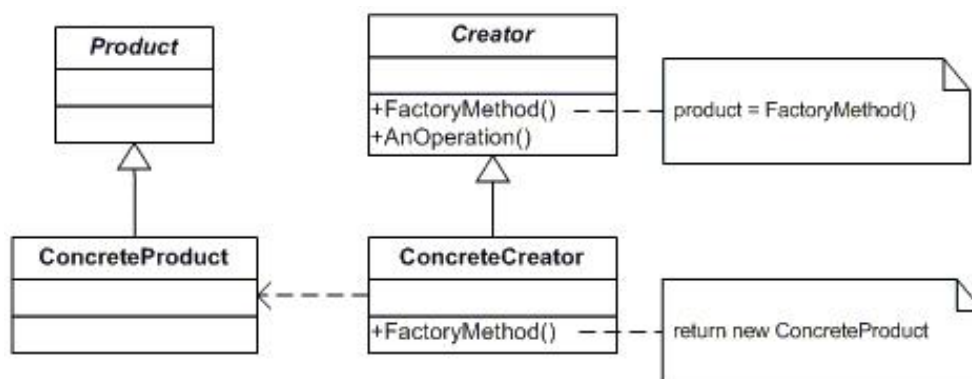
прикладом використання абстрактної фабрики є ADO.NET: существує загальний клас `DbProviderFactory`, здатний створювати об'єкти типів `DbConnection`, `DbDataReader`, `DbAdapter` та ін.; існують реалізації цих фабрик і об'єктів - `SqlProviderFactory`, `SqlConnection`, `SqlDataReader`, `SqlAdapter` і так далі. Відповідно, якщо додатку необхідно працювати з різними базами даних (чи потрібна така можливість), то досить використати базові реалізації (`Db.`) і под-ставити відповідну фабрику у момент ініціалізації фабрики (`Factory = new SqlProviderFactory()`). Цей шаблон передусім структурує знання про схожі об'єкти (що називаються сімействами, як класи для доступу до БД) і створює можливість взаємозаміни різних сімейств (робота з Oracle ведеться також, як і робота з SQL Server). Проте, при використанні такої схеми укр. незручно розширювати фабрику - для додавання нового методу у фабрику необхідно додати його в усіх фабриках і створити відповідні класи, що створюються цим методом.

Переваги та недоліки:

- + Гарантує поєднання створюваних продуктів.
- + Звільняє клієнтський код від прив'язки до конкретних класів продукту.
- + Реалізує *принцип відкритості/закритості*.
- Вимагає наявності всіх типів продукту в кожній варіації.
- Ускладнює код програми внаслідок введення великої кількості додаткових класів.

Шаблон «Factory Method»

Структура:



Призначення:

Шаблон "фабричний метод" визначає інтерфейс для створення об'єктів певного базового типу. Це зручно, коли хочеться додати можливість створення об'єктів не базового типу, а деякого дочірнього. Фабричний метод у такому разі є зачіпкою для впровадження власного конструктора об'єктів. Основна ідея полягає саме в заміні об'єктів їх підтипами, що при цьому зберігає ту ж функціональність; інша частина поведінки об'єктів не є інтерфейсною (`AnOperation`) і дозволяє взаємодіяти із створеними об'єктами як з об'єктами базового типу. Тому шаблон "фабричний метод" носить ще назву "Віртуальний конструктор".

Розглянемо простий приклад. Нехай наше застосування працює з мережевими драйвер-мі і використовує клас `Packet` для зберігання даних, що передаються в мережу. Залежно від використовуваного протоколу, існує два перевантаження - `TcpPacket`, `UdpPacket`. І відповідно два створюючі об'єкти (`TcpCreator`, `UdpCreator`) з фабричним методом (який створює відповідні реалізації).

Проте базова функціональність (передача пакету, прийом пакету, заповнення пакету даними) нічим не відрізняється один від одного, відповідно поміщається у базовий клас `PacketCreator`. Таким чином поведінка системи залишається тим же, проте з'являється

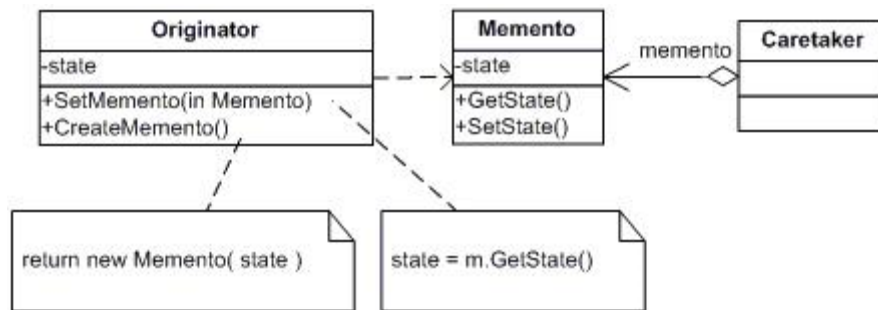
можливість підстановки власних об'єктів в процес створення і роботи з пакетами.

Переваги та недоліки:

- + Позбавляє клас від прив'язки до конкретних класів продуктів.
- + Виділяє код виробництва продуктів в одне місце, спрощуючи підтримку коду.
- + Спрощує додавання нових продуктів до програми.
- Може призвести до створення великих паралельних ієрархій класів.

Шаблон «Memento»

Структура:



Призначення:

Шаблон використовується для збереження і відновлення стану об'єктів без порушення інкапсуляції. Об'єкт "мементо" служить виключно для збереження змін над початковим об'єктом (Originator). Лише початковий об'єкт має можливість зберігати і отримувати стан об'єкту "мементо" для власних цілей, цей об'єкт є "порожнім" для кого-небудь ще. Об'єкт Caretaker використовується для передачі і зберігання мементо об'єктів в системі. Таким чином вдається досягти наступних цілей:

- Зберігання стану повністю відділяється від початкових об'єктів, що полегшує їх реалізацію;
- Передача об'єктів мементо лягає на плечі Caretaker об'єктів, що дозволяє гнучкіше управляти

станами об'єктів і спростити дизайн класів початкових об'єктів;

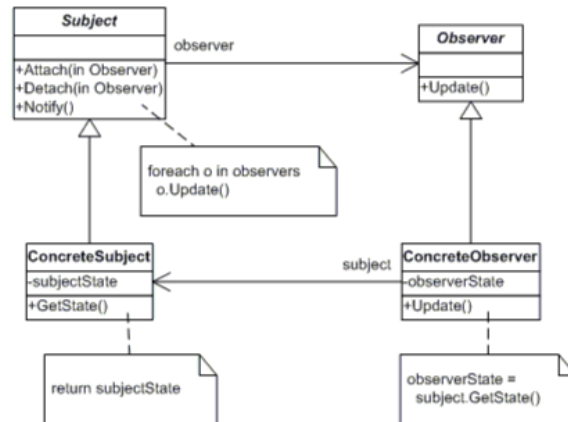
- Збереження і відновлення стану реалізовані у вигляді двох простих методів і є закритими для кого-небудь ще окрім початкових об'єктів, таким чином не порушуючи інкапсуляцію.
- Шаблон "мементо" дуже зручно використати разом з шаблоном "команда" для реалізації "скасовних" дій - дані про дію зберігаються в мементо, а команда має можливість вважати і відновити початкове положення відповідних об'єктів.

Переваги та недоліки:

- + Не порушує інкапсуляцію вихідного об'єкта.
- + Спрощує структуру вихідного об'єкта. Не потрібно зберігати історію версій свого стану.
- Вимагає багато пам'яті, якщо клієнти дуже часто створюють знімки.
- Може спричинити додаткові витрати пам'яті, якщо об'єкти, що зберігають історію, не звільняють ресурси, зайняті застарілими знімками.

Шаблон «Observer»

Структура:



Призначення:

Шаблон визначає залежність "один-ко-многим" таким чином, що коли один об'єкт змінює власний стан, усі інші об'єкти отримують про це сповіщення і мають можливість змінити власний стан також.

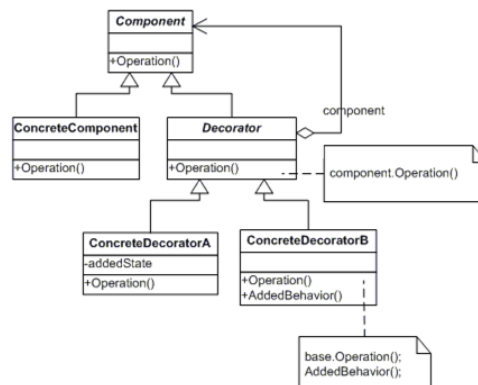
Розглянемо цей шаблон на прикладі. Припустимо, є деяка банківська система і декілька користувачів переглядають баланс на рахунку пана І. У цей момент пан І. кладе на свій рахунок деяку суму, яка міняє загальний баланс. Кожен з користувачів, що переглядали баланс, отримує про це звістку (для користувачів ця звістка може бути прозорою - просто зміна цифр, або попередження про те, що баланс змінився). Раніше неможливі дії для користувачів (переведення до іншої категорії клієнтів і тому подібне) стають доступними. Цей шаблон дуже широко поширений в шаблоні MVVM і механізмі "прив'язок" (bindings) в WPF і частково в WinForms. Інша назва шаблону - підписка/розсилка. Кожен з оглядачів власноручно підписується на зміни конкретного об'єкту, а об'єкти зобов'язані сповіщати своїх передплатників про усі свої зміни (на даний момент конкретних механізмів автоматичного сповіщення про зміну стану в .NET мовах не існує).

Переваги та недоліки:

- + Ви можете підписувати і відписувати одержувачів «на льоту».
- + Видавці не залежать від конкретних класів підписників і навпаки.
- + Реалізує *принцип відкритості/закритості*.
- Підписники сповіщуються у випадковій послідовності.

Шаблон «Decorator»

Структура:



Призначення:

Шаблон призначений для динамічного додавання функціональних можливостей об'єкту під час роботи програми. Декоратор деяким чином "обертає" (агрегація) початковий об'єкт зі збереженням його функцій, проте дозволяє додати додаткові дії. Такий шаблон надає гнучкіший спосіб зміни поведінки об'єкту чим просте спадкоємство, оскільки початкова функціональність зберігається в повному об'ємі. Більше того, таку поведінку можна застосовувати до окремих об'єктів, а не до усієї системи в цілому.

Простим прикладом є накладення смуги прокрутки до усіх візуальних елементів. Кожен об'єкт, який може прокручуватися, обертається в "прокручуваному" елементі, і при необхідності з'являється смуга прокрутки. Початкові функції елементу (наприклад, рядки статусу) залишаються незмінними.

Переваги та недоліки:

- + Дозволяє мати кілька дрібних об'єктів, замість одного об'єкта «на всі випадки життя».
- + Дозволяє додавати обов'язки «на льоту».
- + Більша гнучкість, ніж у спадкування.
- Велика кількість крихлих класів.
- Важко конфігурувати об'єкти, які загорнуто в декілька обгортки одночасно

Реалізація не менше 3-х класів відповідно до обраної теми

Структура проекту з реалізованими класами зображена на рисунку 1:

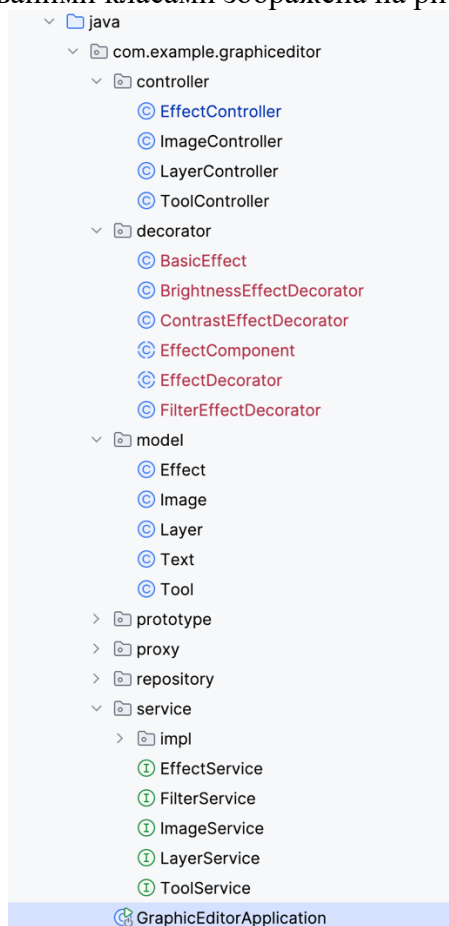


Рис.1 Структура проекту з реалізованими класами

У ході лабораторної роботи була реалізована система графічного редактора, яка використовує патерн decorator для динамічного додавання нових ефектів до зображень без зміни існуючого коду. Основна мета — забезпечити модульність і гнучкість при створенні та комбінуванні різних ефектів обробки.

Опис реалізації:

1. Базовий компонент (EffectComponent)

Це абстрактний клас, який визначає основний метод:

- String applyEffect() — застосовує ефекти до зображення або шару.

2. Базовий клас ефекту (BasicEffect)

Клас BasicEffect є базовою реалізацією EffectComponent, яка виконує мінімальну обробку:

- Метод applyEffect() повертає "Base effect applied."

3. Абстрактний клас декоратора (EffectDecorator)

Абстрактний клас EffectDecorator наслідує EffectComponent і зберігає посилання на інший компонент (ефект), дозволяючи накладати додаткові ефекти:

- Конструктор приймає EffectComponent і передає виклики до базового ефекту через super.applyEffect().

4. Декоратори ефектів

Декоратори реалізують розширення функціональності базового компонента. У кожному декораторі перевизначений метод applyEffect() додає нову логіку до ефектів, зберігаючи їхню черговість.

BrightnessEffectDecorator: Додає ефект налаштування яскравості.

- Повертає результат базового ефекту з додаванням рядка " Brightness adjusted."

ContrastEffectDecorator: Додає ефект налаштування контрасту.

- Повертає результат базового ефекту з додаванням рядка " Contrast adjusted."

FilterEffectDecorator: Додає ефект застосування фільтру.

- Повертає результат базового ефекту з додаванням рядка " Filter applied."

5. Контролер

Реалізовано REST-контролер із методом `applyCustomEffect`, який дозволяє динамічно додавати та комбінувати ефекти за допомогою HTTP-запиту.

Використання патерну `decorator` дозволило створити гнучку систему обробки зображень, що легко розширюється. Користувач має змогу комбінувати ефекти та динамічно змінювати їх під час виконання програми, що робить систему більш функціональною та зручною у використанні.

Реалізація шаблону за обраною темою

У ході роботи було реалізовано систему графічного редактора, в якій застосовано патерн `decorator` для створення гнучкої архітектури роботи з ефектами зображення.

Основна мета використання цього патерна — забезпечення можливості динамічного додавання нових ефектів до базового зображення без зміни початкового коду класів. Це дозволяє легко розширювати функціональність, зберігаючи принципи модульності та повторного використання коду.

Опис реалізації:

1. Клас *BasicEffect*

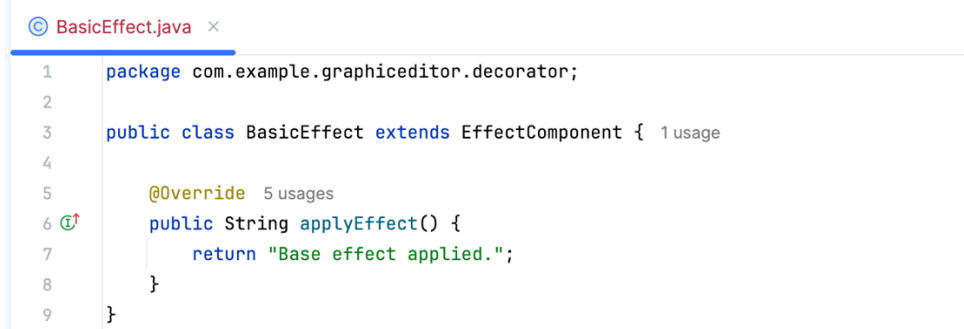
Клас `BasicEffect` реалізує базовий ефект, який є основою для всіх інших графічних ефектів.

Основні функції:

- Базова реалізація методу `applyEffect()`, що повертає повідомлення про застосування основного ефекту.
- Слугує початковою точкою для створення ланцюга ефектів через патерн `decorator`.

Можливості:

- Визначає єдиний базовий інтерфейс для роботи з ефектами.
- Може використовуватись як самостійний компонент або бути декорованим іншими ефектами.

A screenshot of an IDE showing the implementation of the BasicEffect class. The code is in Java and defines a package, imports a base class, and implements an abstract method. The package is 'com.example.graphiceditor.decorator'. The class 'BasicEffect' extends 'EffectComponent'. It has a single method 'applyEffect()' which returns the string 'Base effect applied.'.

```
1 package com.example.graphiceditor.decorator;
2
3 public class BasicEffect extends EffectComponent {
4     1 usage
5     @Override 5 usages
6     public String applyEffect() {
7         return "Base effect applied.";
8     }
9 }
```

Рис.2 Реалізація класу `BasicEffect`

2. Клас *EffectComponent*

`EffectComponent` — це абстрактний клас, який задає структуру для всіх ефектів.

Основні функції:

- Визначає абстрактний метод `applyEffect()`, який реалізується в похідних класах.
- Забезпечує єдиний інтерфейс для всіх компонентів та декораторів ефектів.

Можливості:

- Дає змогу створювати ієрархію компонентів для додавання ефектів у вигляді ланцюга.

```

1 package com.example.graphiceditor.decorator;
2
3 public abstract class EffectComponent { 8 usages 5 inheritors
4     public abstract String applyEffect(); 5 usages 5 implementations
5 }

```

Рис.3 Реалізація класу EffectComponent

3. Клас EffectDecorator

EffectDecorator — це абстрактний клас, що виступає основою для створення конкретних декораторів.

Основні функції:

- Має посилання на об'єкт типу EffectComponent, що дозволяє поєднувати ефекти в ланцюг.
- Реалізує метод applyEffect(), делегуючи його виконання об'єкту, переданому через конструктор.

Можливості:

- Забезпечує динамічне розширення функціональності базового ефекту.
- Декоратори можна комбінувати для створення складних ефектів.

```

1 package com.example.graphiceditor.decorator;
2
3 public abstract class EffectDecorator extends EffectComponent { 3 usages 3 inheritors
4     protected EffectComponent effectComponent; 2 usages
5
6     public EffectDecorator(EffectComponent effectComponent) { 3 usages
7         this.effectComponent = effectComponent;
8     }
9
10    @Override 5 usages 3 overrides
11    public String applyEffect() {
12        return effectComponent.applyEffect();
13    }
14 }

```

Рис.4 Реалізація класу EffectDecorator

4. Класи-декоратори: BrightnessEffectDecorator, ContrastEffectDecorator, FilterEffectDecorator

Ці класи є конкретними реалізаціями декораторів, які додають специфічну функціональність до базового ефекту:

1. BrightnessEffectDecorator

- Змінює яскравість зображення.
- У методі applyEffect() викликає базовий ефект і додає інформацію про зміну яскравості.



```

1 package com.example.graphiceditor.decorator;
2
3 public class BrightnessEffectDecorator extends EffectDecorator { 1 usage
4
5     public BrightnessEffectDecorator(EffectComponent effectComponent) { 1 usage
6         super(effectComponent);
7     }
8
9     @Override 5 usages
10    public String applyEffect() {
11        return super.applyEffect() + " Brightness adjusted.";
12    }
13 }

```

Рис.5 Реалізація класу BrightnessEffectDecorator

2. ContrastEffectDecorator

- Змінює контрастність зображення.
- Розширює функціональність, додаючи логіку корекції контрастності.



```

1 package com.example.graphiceditor.decorator;
2
3 public class ContrastEffectDecorator extends EffectDecorator { 1 usage
4
5     public ContrastEffectDecorator(EffectComponent effectComponent) { 1 usage
6         super(effectComponent);
7     }
8
9     @Override 5 usages
10    public String applyEffect() {
11        return super.applyEffect() + " Contrast adjusted.";
12    }
13 }

```

Рис.6 Реалізація класу ContrastEffectDecorator

3. FilterEffectDecorator

- Застосовує фільтр до зображення.
- У методі applyEffect() викликає всі попередні ефекти й додає інформацію про застосування фільтра.



```

1 package com.example.graphiceditor.decorator;
2
3 public class FilterEffectDecorator extends EffectDecorator { 1 usage
4
5     public FilterEffectDecorator(EffectComponent effectComponent) { 1 usage
6         super(effectComponent);
7     }
8
9     @Override 5 usages
10    public String applyEffect() {
11        return super.applyEffect() + " Filter applied.";
12    }
13 }

```

Рис.7 Реалізація класу FilterEffectDecorator

Можливості:

- Декоратори можна комбінувати в довільному порядку, забезпечуючи гнучкість.
- Кожен декоратор додає свою унікальну поведінку, не змінюючи існуючий код.

Основні можливості та переваги реалізації:

1. Динамічне додавання функціональності:
Завдяки патерну decorator можна додавати нові ефекти до базового об'єкта під час виконання програми.
2. Гнучкість у роботі з ефектами:
Користувач може комбінувати ефекти у різному порядку залежно від потреб.
3. Простота розширення:
Для додавання нового ефекту достатньо створити новий клас-декоратор, не змінюючи існуючий код.
4. Збереження принципів SOLID:
 - Принцип відкритості/закритості (O): Код можна розширювати, але не змінювати.
 - Принцип єдиної відповідальності (S): Кожен декоратор відповідає лише за свою частину логіки.

Патерн decorator дозволив створити гнучку та розширювану систему для роботи з ефектами графічного редактора. Реалізація цього підходу забезпечила:

1. Легку інтеграцію нових функцій.
2. Можливість комбінувати ефекти у різних варіаціях без зміни базового коду.
3. Модульність та відповідність принципам об'єктно-орієнтованого програмування.

Таким чином, використання decorator значно спрощує підтримку та розширення системи графічного редактора.

Діаграма класів для патерну Decorator

Діаграма класів, яка реалізовує патерн decorator зображена на рисунку 8:

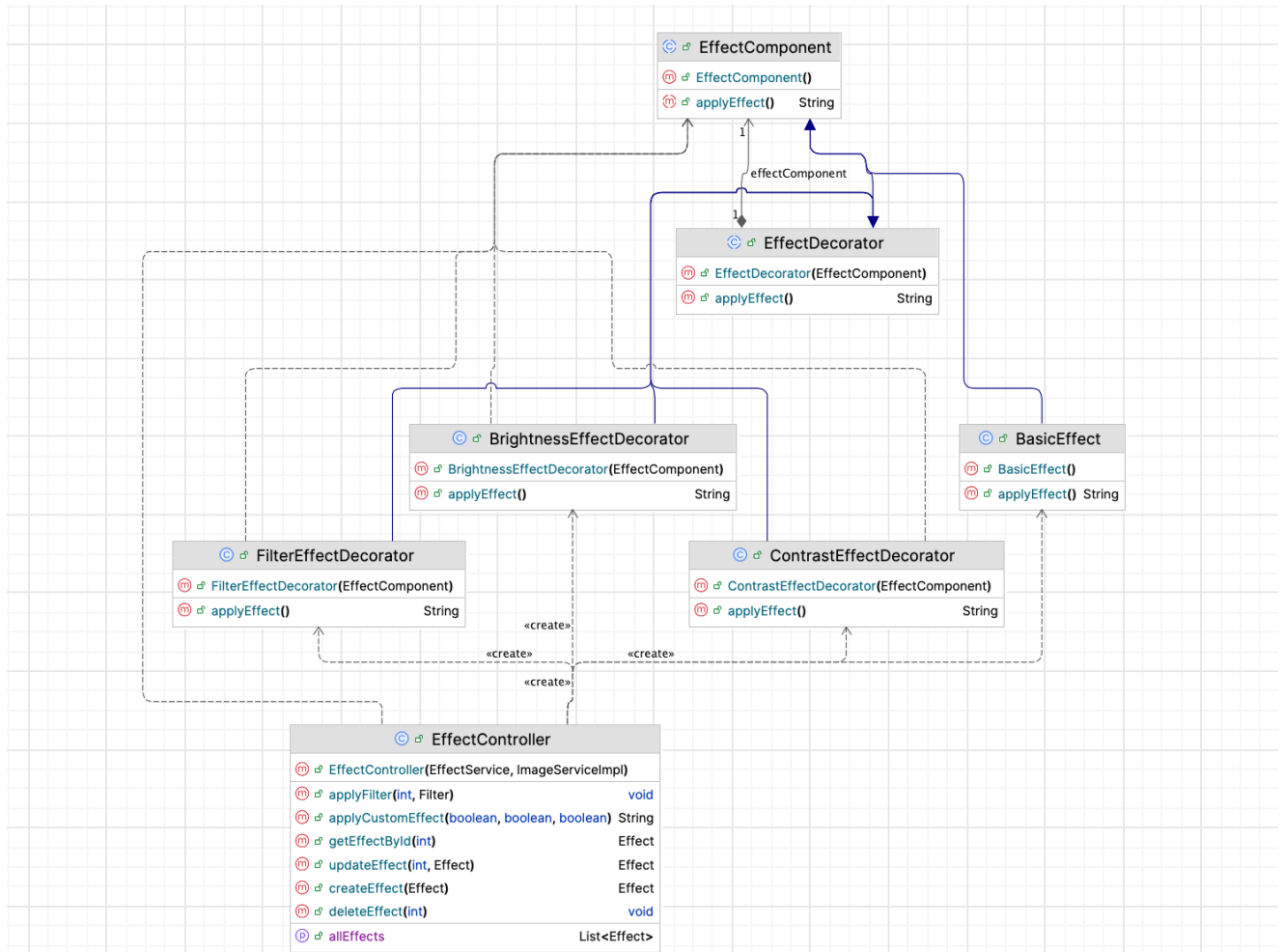


Рис. 8 Діаграма класів для патерну Decorator

Ця діаграма класів представляє реалізацію патерну Decorator для додавання ефектів до зображень у графічному редакторі.

Основні компоненти:

1. EffectComponent (Інтерфейс)

Визначає загальний інтерфейс для всіх ефектів.

Містить метод `applyEffect()`, який повертає рядок (ймовірно, назву чи опис застосованого ефекту).

2. BasicEffect (Клас)

Базова реалізація інтерфейсу **EffectComponent**.

Визначає стандартну поведінку для застосування ефектів.

Реалізує метод `applyEffect()`.

3. EffectDecorator (Абстрактний клас)

Реалізує інтерфейс **EffectComponent** та діє як базовий клас для конкретних декораторів.

Містить посилання на **EffectComponent** (композиція), що дозволяє динамічно обертати інші об'єкти цього типу.

Перевизначає метод `applyEffect()`, делегуючи виклик обгорнутому компоненту.

Конкретні декоратори:

1. `BrightnessEffectDecorator`

Наслідує `EffectDecorator`.

Додає логіку зміни яскравості до методу `applyEffect()`.

2. `ContrastEffectDecorator`

Наслідує `EffectDecorator`.

Додає логіку зміни контрастності до методу `applyEffect()`.

3. `FilterEffectDecorator`

Наслідує `EffectDecorator`.

Реалізує метод `applyEffect()` для додавання спеціальних фільтрів до зображення.

Контролер:

`EffectController` (Клас)

Координує використання ефектів та їх застосування до зображень.

Методи:

- `applyFilter(int, Filter)`: застосовує певний фільтр.
- `applyCustomEffect(boolean, boolean, boolean)`: застосовує кастомні ефекти (ймовірно, яскравість, контраст тощо).
- `getEffectById(int)`: отримує ефект за ідентифікатором.
- `updateEffect(int, Effect)`: оновлює ефект.
- `createEffect(Effekt)`: створює новий ефект.
- `deleteEffect(int)`: видаляє ефект.
- `allEffects`: повертає список усіх ефектів.

Взаємодія:

`BasicEffect` слугує базовим компонентом.

`EffectDecorator` забезпечує основу для динамічного розширення функціоналу за допомогою конкретних декораторів (`BrightnessEffectDecorator`, `ContrastEffectDecorator`, `FilterEffectDecorator`).

`EffectController` управляє створенням, використанням і видаленням ефектів, працюючи з об'єктами типу `EffectComponent`.

Таким чином, патерн `Decorator` дозволяє динамічно додавати різні ефекти до зображення, уникаючи множинного наслідування.

Проблема, яку допоміг вирішити шаблон `Decorator`

Шаблон `decorator` у цьому графічному редакторі допоміг вирішити кілька важливих проблем, пов'язаних із роботою з ефектами для зображень.

1. Динамічне додавання функціональності без зміни існуючого коду

Шаблон `Decorator` дозволив створювати нові ефекти для зображень (наприклад, яскравість, контраст, фільтри) без необхідності змінювати код базових класів.

- Клас `BasicEffect` реалізує базовий функціонал (`applyEffect()`), який можна розширити за допомогою декораторів.
- Наприклад, клас `BrightnessEffectDecorator` додає логіку зміни яскравості, тоді як `ContrastEffectDecorator` додає зміну контрасту.

Це дозволяє легко додавати нові ефекти (навіть під час виконання програми) без втручання в існуючі класи.

2. Комбінування різних ефектів

Завдяки рекурсивній природі шаблону, можна комбінувати різні ефекти у довільній послідовності.

Це забезпечує гнучкість і дає можливість комбінувати будь-які ефекти без створення нових класів для кожної комбінації.

3. Заміна жорсткої ієрархії наслідування

Без використання шаблону decorator можна було б створити окремий клас для кожної можливої комбінації ефектів, наприклад:

- BrightnessAndContrastEffect
- FilterAndBrightnessEffect
- BrightnessAndContrastAndFilterEffect

Це призвело б до величезної кількості класів, які важко підтримувати. Decorator вирішує цю проблему, дозволяючи динамічно «обгортати» ефекти.

4. Гнучка модифікація під час виконання

Можемо змінювати, додавати або видаляти ефекти під час роботи програми.

Тепер до вже існуючої послідовності ефектів додано ще один.

5. Простота тестування

Кожен конкретний декоратор (BrightnessEffectDecorator, ContrastEffectDecorator, FilterEffectDecorator) легко тестувати окремо. Це робить систему більш надійною та спрощує розробку.

Шаблон decorator оптимізував роботу з ефектами в редакторі:

- Спрощено додавання нових ефектів.
- Реалізовано гнучке комбінування та налаштування ефектів без дублювання коду.
- Уникнуто жорсткої ієрархії наслідування.
- Система стала розширюваною, модульною і зручною для підтримки.

Переваги застосування патерну Decorator

Використання патерну decorator у моєму графічному редакторі має такі переваги:

1. Гнучке розширення функціональності

- Decorator дозволяє динамічно додавати нові ефекти до зображень (наприклад, яскравість, контраст, фільтри) без змін у коді базових класів.
- Замість створення окремих класів для кожної можливої комбінації ефектів, можна комбінувати їх у довільному порядку, обгортаючи об'єкти один в одного.

2. Зменшення складності ієрархії наслідування

- Замість створення складної ієрархії класів для кожного варіанта ефекту (BrightnessAndContrastEffect, FilterAndBrightnessEffect тощо), кожен ефект реалізується як окремий декоратор.
- Це значно спрощує підтримку коду та його модифікацію.

3. Комбінування ефектів

- Ефекти можна легко комбінувати.
- Така гнучкість дозволяє застосовувати будь-які ефекти у потрібній послідовності, що робить систему максимально адаптивною.

4. Динамічне додавання/видалення функціональності

- Під час виконання програми можна додавати або прибирати ефекти без повторного створення об'єктів.
- Це забезпечує зручність у випадках, коли потрібно оперативно змінювати ефекти на шарі зображення.

5. Простота тестування

- Кожен декоратор (BrightnessEffectDecorator, ContrastEffectDecorator, FilterEffectDecorator) працює незалежно. Це дозволяє тестувати кожен ефект окремо, спрощуючи виявлення помилок.

6. Розширюваність

- Для додавання нового ефекту достатньо створити новий клас-декоратор, який наслідує EffectDecorator, і перевизначити метод applyEffect().
- Наприклад, для додавання ефекту насиченості потрібно лише додати новий декоратор SaturationEffectDecorator.

7. Збереження базового функціоналу

- Патерн зберігає базовий функціонал компонентів (наприклад, BasicEffect), дозволяючи модифікувати його лише за допомогою декораторів. Це забезпечує інкапсуляцію та захист від небажаних змін.

8. Модульність і повторне використання

- Декоратори можна використовувати повторно у різних частинах проєкту. Наприклад, BrightnessEffectDecorator можна застосувати як до зображень, так і до відео.

Ці переваги зробили графічний редактор більш модульним, гнучким і масштабованим. Завдяки Decorator легко розширювати функціональність, забезпечуючи інтуїтивне управління ефектами без зайвого дублювання коду чи ризику порушення роботи базових класів.

Посилання на GitHub

Посилання на репозиторій: <https://github.com/Ellviraaaaaa/GraphicEditor.git>

Висновок

У даній лабораторній роботі я реалізувала патерн проєктування decorator для графічного редактора, що дозволило динамічно додавати нові функції до об'єктів без зміни їхнього базового коду. Завдяки цьому шаблону вдалося створити гнучку систему для роботи з ефектами зображень, забезпечивши можливість комбінування, додавання та видалення ефектів у довільному порядку.

Застосування Decorator дозволило уникнути створення складної ієрархії наслідування, спростивши структуру програми та полегшивши розширення її функціональності. Це забезпечило модульність, повторне використання коду та простоту підтримки системи, а також зробило процес обробки зображень більш гнучким, масштабованим та ефективним.